Thinking in parallel

How to design algorithms for multi-processor systems

Objectives

- Present algorithms suitable for implementation on parallel computers
  - We emphasize techniques, paradigms, and methods, rather than detailed algorithms for specific applications

Contents

1. Introduce some of the many parallel models and discuss their pros and cons
   - DAGs
   - network models
   - shared-memory models
2. Choose a good framework for designing and analyzing parallel algorithms
   - We will use the PRAM (a famous shared-memory model)
3. Present basic results on the PRAM
   - Basic techniques
   - Lists, trees, searching, merging, and hopefully more
4. Present the very recent Massively Parallel Computing (MPC) model which is more related to the concept of big data

The what's and why of parallel processing

DEFINITION: A **parallel computer** is a collection of processors, typically of the same type, that are interconnected to allow the coordination of their activities and the exchange of data for the purpose of jointly solving a given problem.

Why do we need parallel processing?

- to perform computation faster than with a single processor
- the need for faster solutions and for solving very large problem instances (*big data*)
  - fluid dynamics, weather prediction, modeling and simulating large systems, information processing and extraction, image processing, artificial intelligence, and automated manufacturing

Performances of parallel algorithms

Sequential algorithms

- n is the size of the input
- Worst-case asymptotic bounds on
  - time complexity
  - space complexity

Parallel algorithms

- n is the size of the input
- p is the number of used processors
- Worst case asymptotic bounds on
  - $T_p(n)$: **parallel time** complexity (i.e., worst-case number of time steps needed to solve a problem instance of size n with p processors that are working in parallel)
  - space complexity
  - communication complexity
  - programmability

In this module we focus on time complexity aspects only

Speedup

The **sequential time complexity** $T^*(n)$ of n-size instances of problem Π indicates the time complexity of the best <u>sequential</u> algorithm for Π.

DEFINITION: The *speedup* $S_p(n)$ of a parallel algorithm of (parallel) time complexity $T_p(n)$ for solving n-size instances of Π with p processors is defined as

$$S_p(n) = \frac{T^*(n)}{T_p(n)}$$

OBSERVATION: $S_p(n) \leq p$

GOAL: Obtain $S_p(n) \approx p$

It is common to replace $T^*(n)$ with the asymptotic time complexity of the best-known sequential algorithm

Parallel model requirements

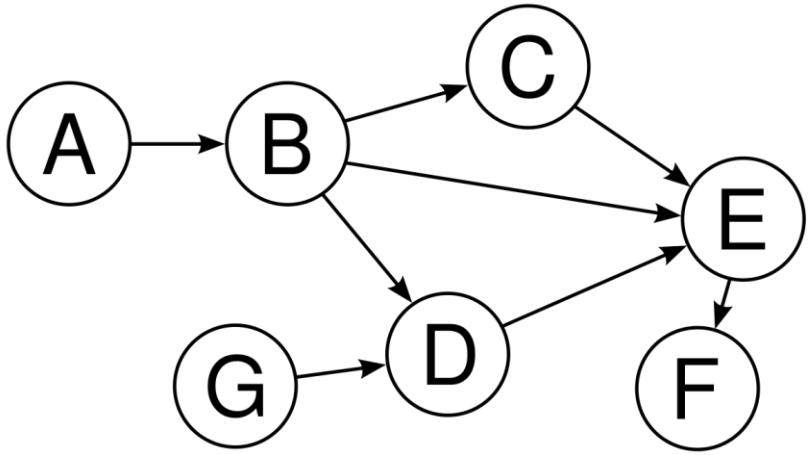
- Simplicity
  - easy to analyze various performance measures
  - results should be as much hardware-independent as possible
- Implementability
  - Parallel algorithms should be easily implementable on a parallel machine
  - Theoretical analysis should give meaningful real performance estimates

A real satisfactory model does not exist

Models

DAG model – computation graph

- nodes represent **operations** (each operation can be executed in $O(1)$ time)
  - The indegree of each node is constant
  - **Sources** represent inputs
  - **Sinks** represent outputs
- edges represent **dependencies** (precedence constraints)

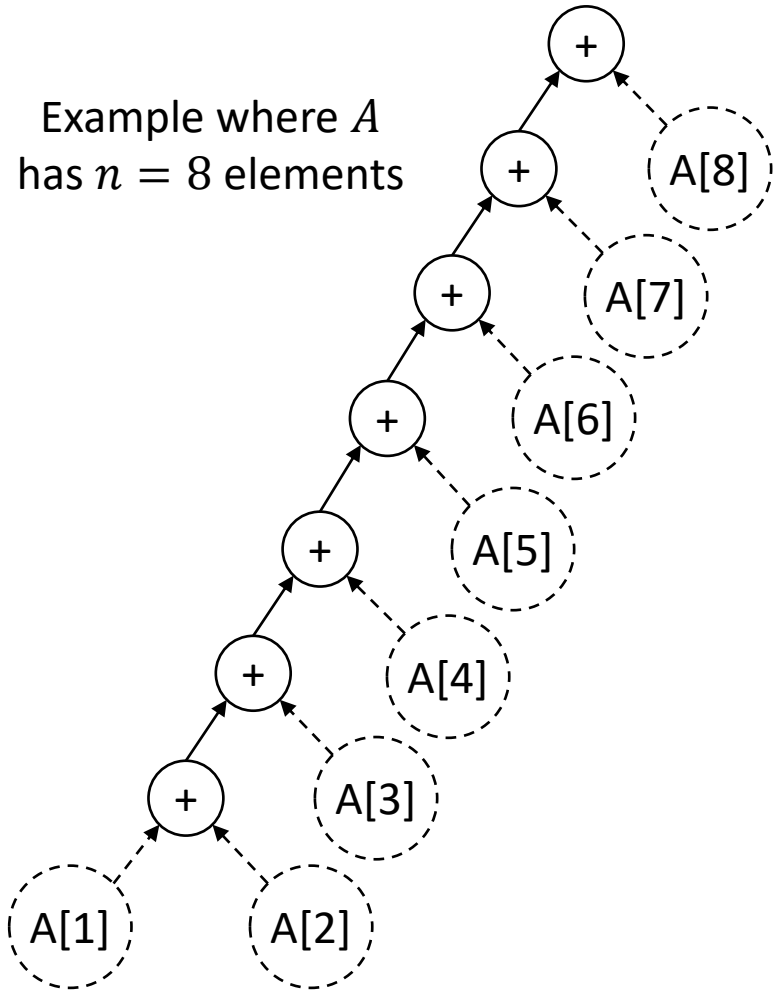


We need A to do B + We need B, C, and D to do E → Transitivity → We need A, B, C, and D to do E

Often used for automatically parallelizing numerical computations

Algorithm: Sum of all n elements of an array A

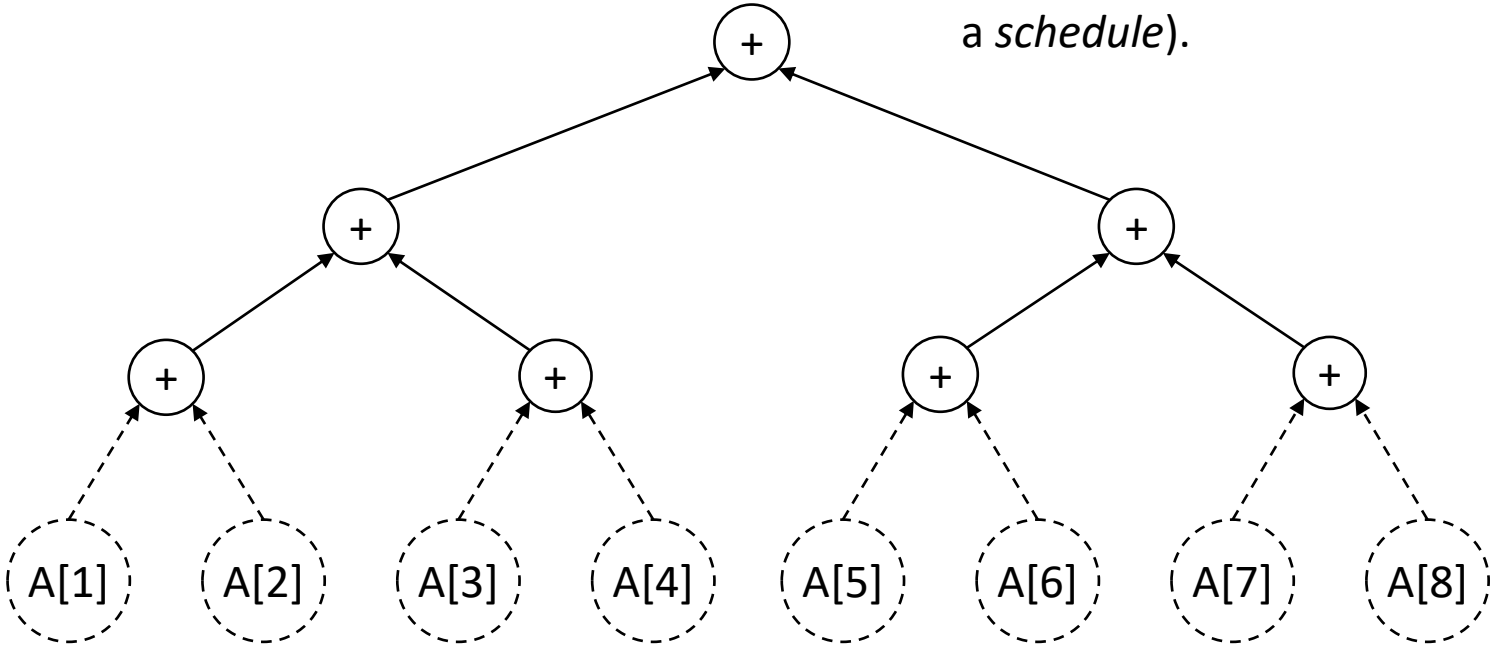
Example where A has $n = 8$ elements



Sequential scheme

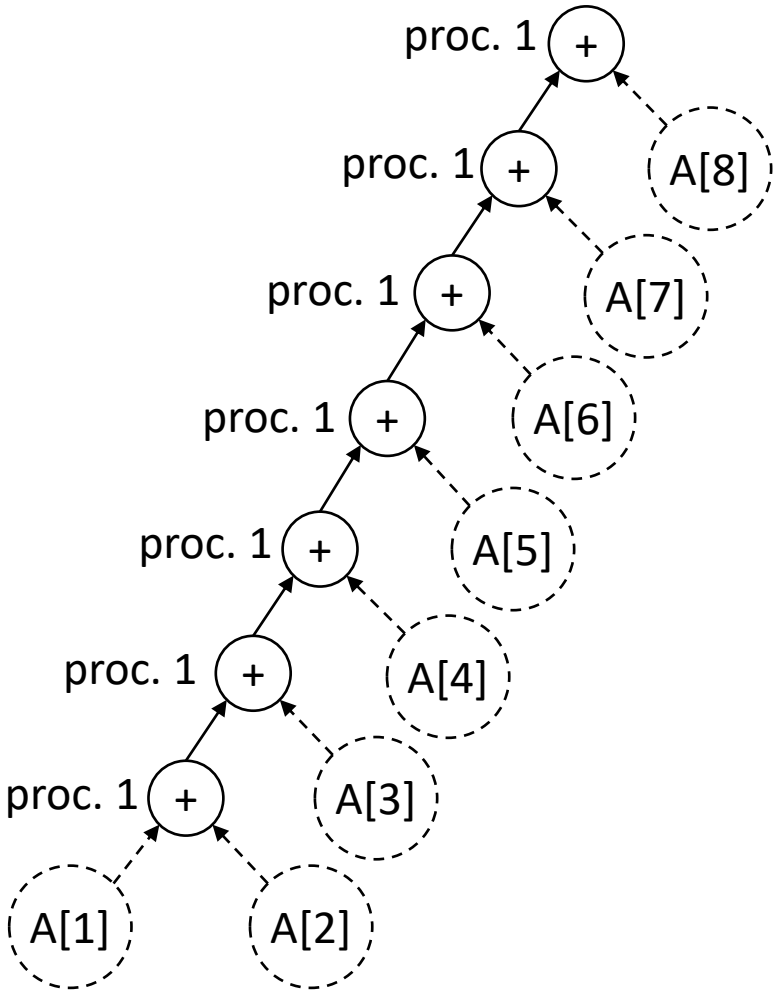
```
sum ← 0
for i = 1 to n do
   sum ← sum + A[i]
```

NOTE: the DAG itself is not a parallel algorithm. We must assign all operations to the p processors (i.e., compute a *schedule*).



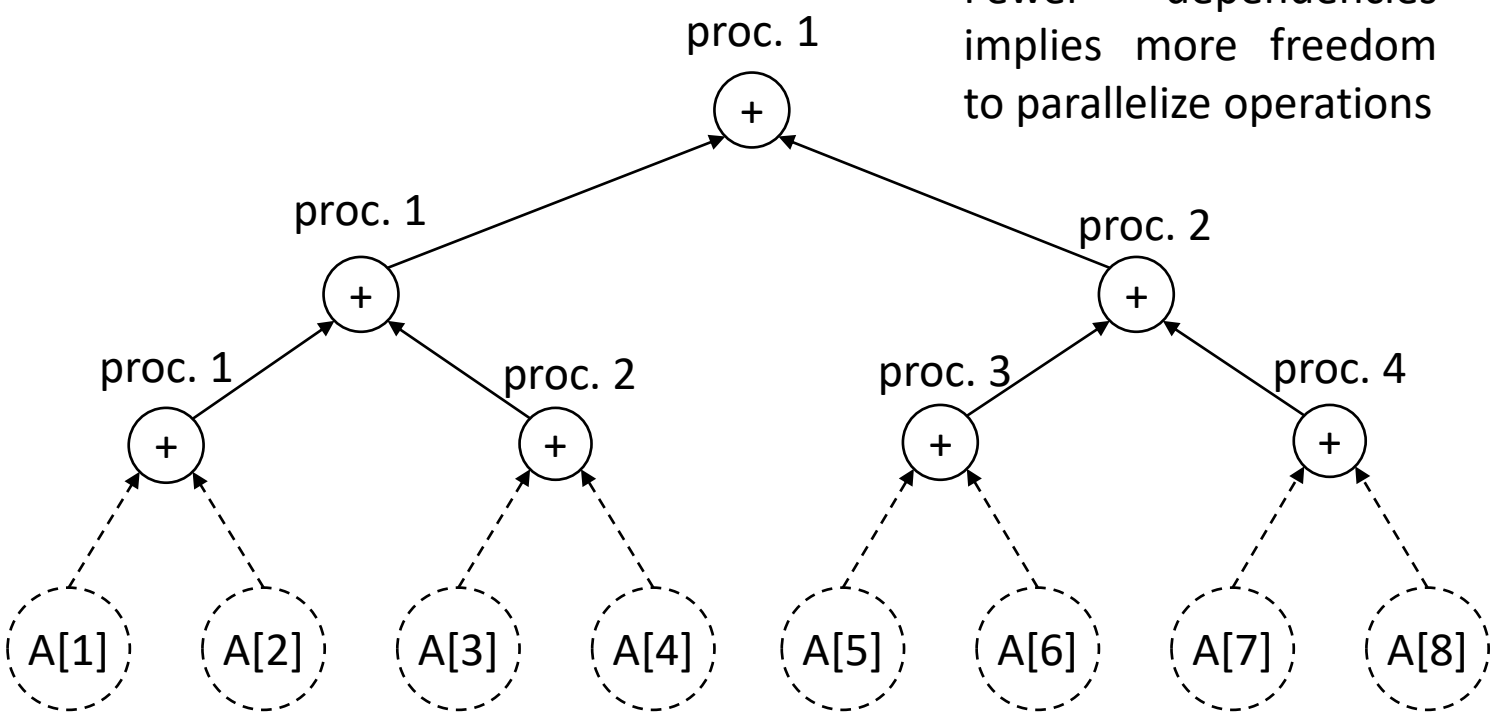
Balanced binary tree scheme

Scheduling: How to assign tasks to processors



Due to the too many dependencies, there is no benefit in using more than 1 processor

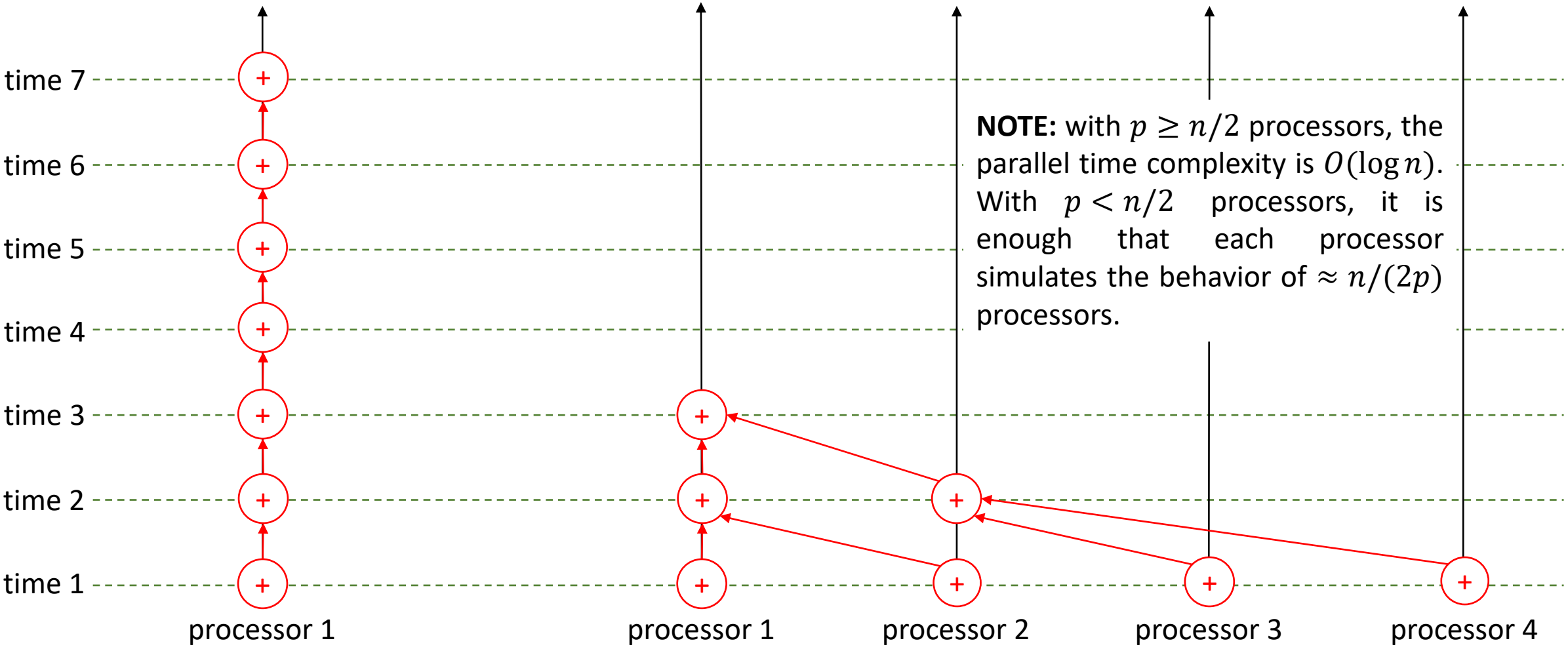
Fewer dependencies implies more freedom to parallelize operations



Sequential scheme (not parallelizable)

Balanced binary tree scheme (parallelizable)

The parallel algorithm: DAG + scheduling



NOTE: with $p \geq n/2$ processors, the parallel time complexity is $O(\log n)$. With $p < n/2$ processors, it is enough that each processor simulates the behavior of $\approx n/(2p)$ processors.

Sequential scheme (not parallelizable)
Parallel time complexity $T_p(n) = O(n)$

Balanced binary tree scheme (parallelizable)

Parallel time complexity $T_p(n) = O\left(\left(\frac{n}{p} + 1\right)\log n\right)$

DAG model: pros and cons

PROS

- completely hardware independent
- simple
- easy to find a good schedule (although finding optimum is NP-hard)

CONS

- useful for a small class of problems
  - If statements cannot be modelled
- some DAGs are difficult to analyze

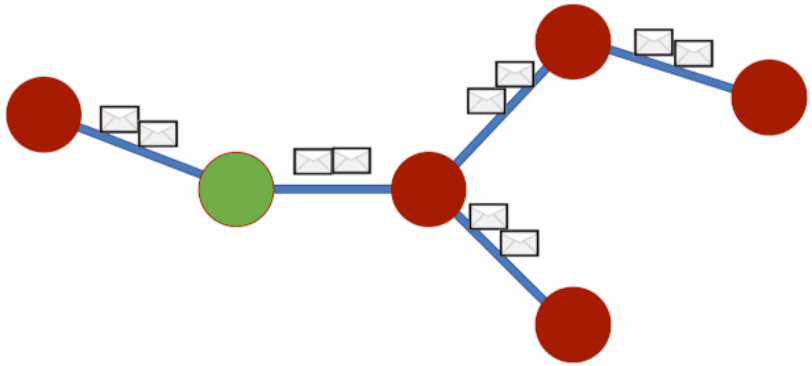
Check you understanding

Let A and B be two $n \times n$ matrices. Consider the standard algorithm to compute the product $C = A \cdot B$, where $C_{i,j}$ is computed using the expression

$$C_{i,j} = \sum_{h=1}^{n} A[i,h] \cdot B[h,j].$$

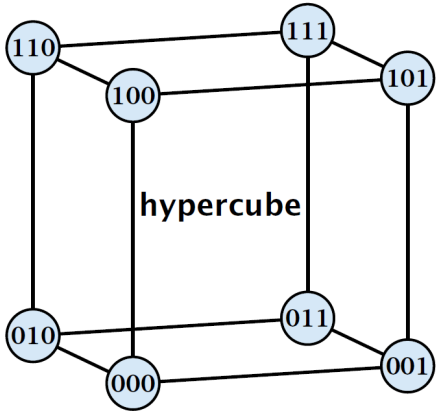
How fast can you compute C with n^3 processors?

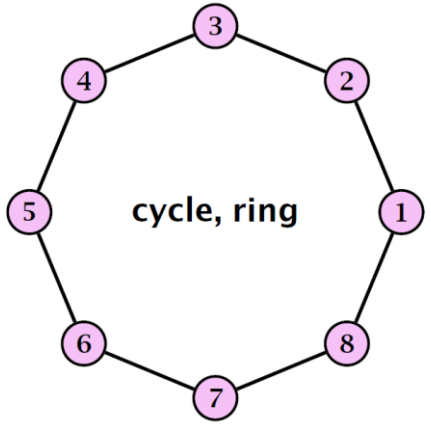
Networks of workstations

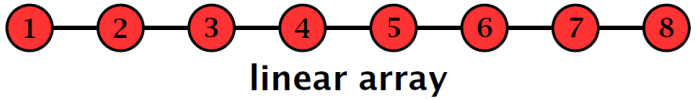


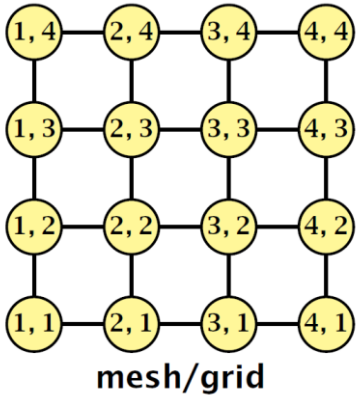
- interconnection network represented by a graph
  - each vertex represents a processor
  - each edge represents a two-way communication link
- each processor has its own **id**
- network can be **synchronous/asynchronous**
- all coordination/communication must be done by explicit **message passing**
  - at any point in time, there is at most one active communication along a link

Typical topologies

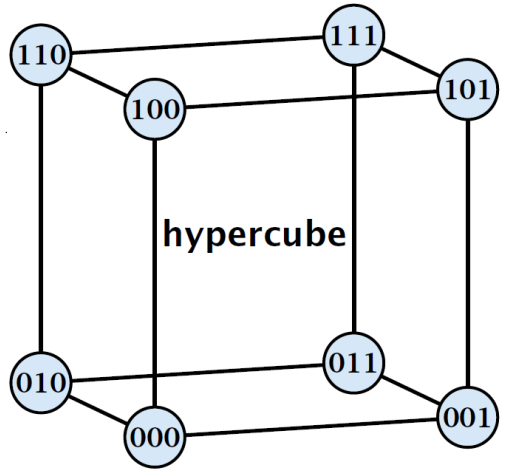








Algorithm: Sum of all n elements of an array A on the Hypercube



/* it stores the sum of all the $n=2^k$ elements of an array A in A[0] */

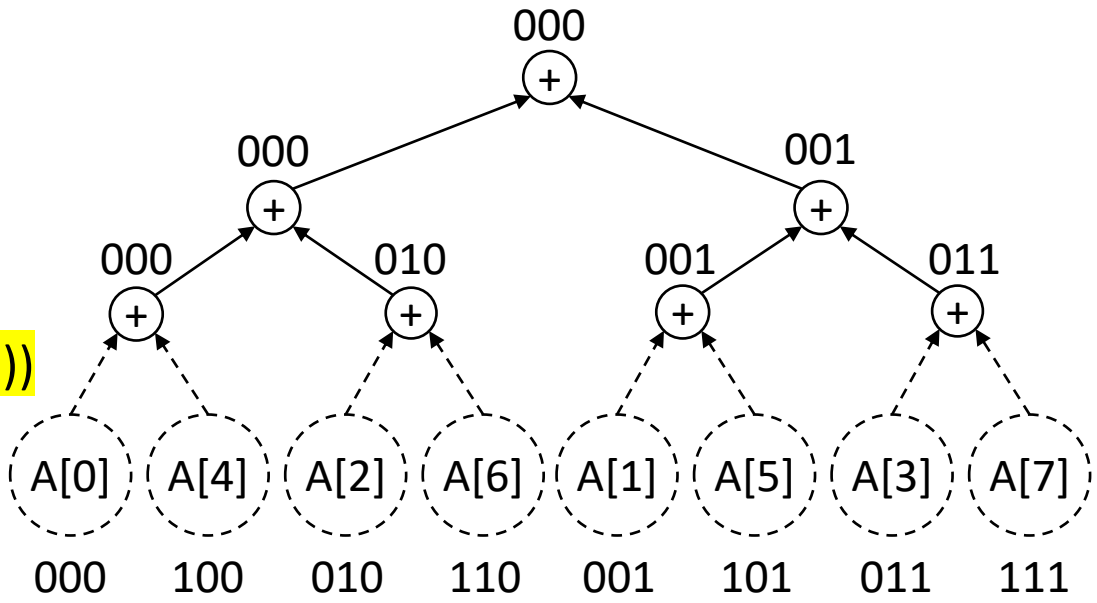
/* We use n processors with binary ids from 0 to n-1 */

/* We assume that A[i] is stored in the local memory of processor with id i */

/* complement(i,h) complements the h-th bit of the binary representation of i */

/* This is the code executed by a generic processor (id denotes the identifier) */

for h = k **downto** 1 **do**
  if $2^{h-1} < id \leq 2^h$ **then**
    send_message(A[id], complement(id, h))
  else if $0 \leq id < 2^{h-1}$
    value ← receive_message(m, complement(id, h))
    A[id] ← A[id] + value



How to measure the parallel time complexity

- Count the number of time steps needed: in our example is $T_n(n) = O(\log n)$
- Count the communication overhead, which depends on
  - *Latency* λ: time to establish the connection
  - *Bandwidth* β: 1/ β time needed to transmit a unit of data
  - *Length of the routing path*: in our example all paths are of length 1
- In our example
  - the number of time steps is $O(k)$
  - the communication overhead of a single processor can be derived from the following
    - At each time step, each processor sends at most 1 message with 1 unit of data → λ+1/β
    - Processor 0 receives k messages, each with 1 unit of data → k(λ+1/β)
    - Processor $i > 0$ receives $k - \lfloor \log_2 i \rfloor$ −1 messages, each with 1 unit of data → $(k - \lfloor \log_2 i \rfloor - 1)$($\lambda$+1/$\beta$)

Networks of workstations: pros and cons

PROS

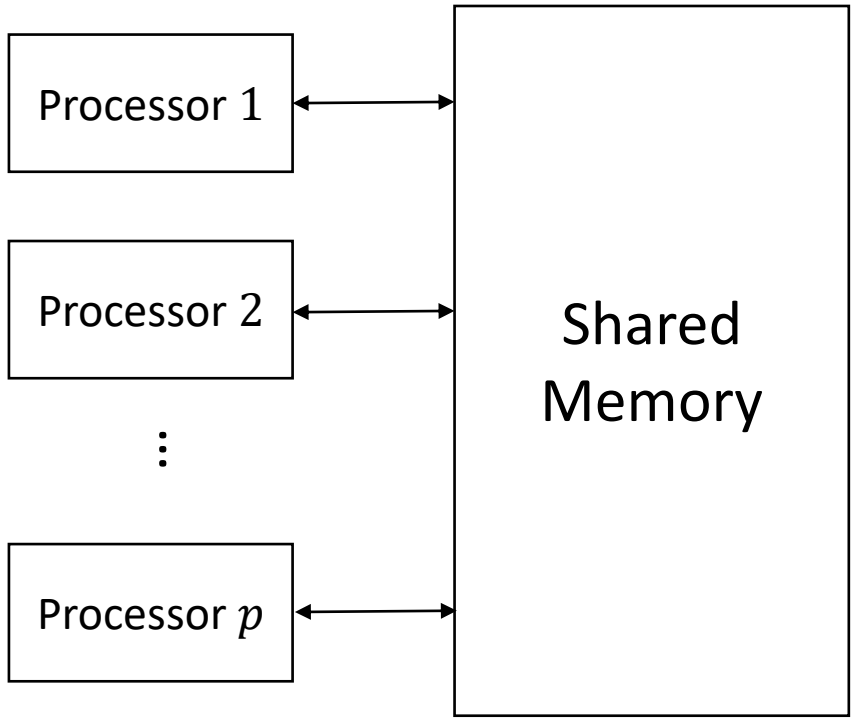
- The asynchronous model is realistic
- There also exist synchronized versions

CONS

- Algorithms are difficult to develop and analyze
  - One must ensure that, at any point in time, there is at most one active communication along a link
  - routing issues
  - lots of low-level communication
- Algorithms for a network topology cannot be generalized easily to other network topologies

The shared-memory model

- p processors, where each processor
  - has its own local memory and executes its own program
  - is uniquely identified by its own **id** in $\{1, \dots, p\}$, which is stored in the local memory of the processor
- The system is **synchronous/asynchronous**
- Processors exchange data through the **shared memory** (a.k.a. **global memory**)
  - global-read(X,y): reads the value of the shared memory variable X and writes it in the local memory variable y
  - global-write(x,Y): reads the value of the local memory variable x and writes it in the shared memory variable Y



Types of shared-memory models

Are concurrent read/write operations on a single memory location allowed in shared memory?

- **EREW**: Exclusive-Read-Exclusive-Write

  (no simultaneous access to a single memory location)

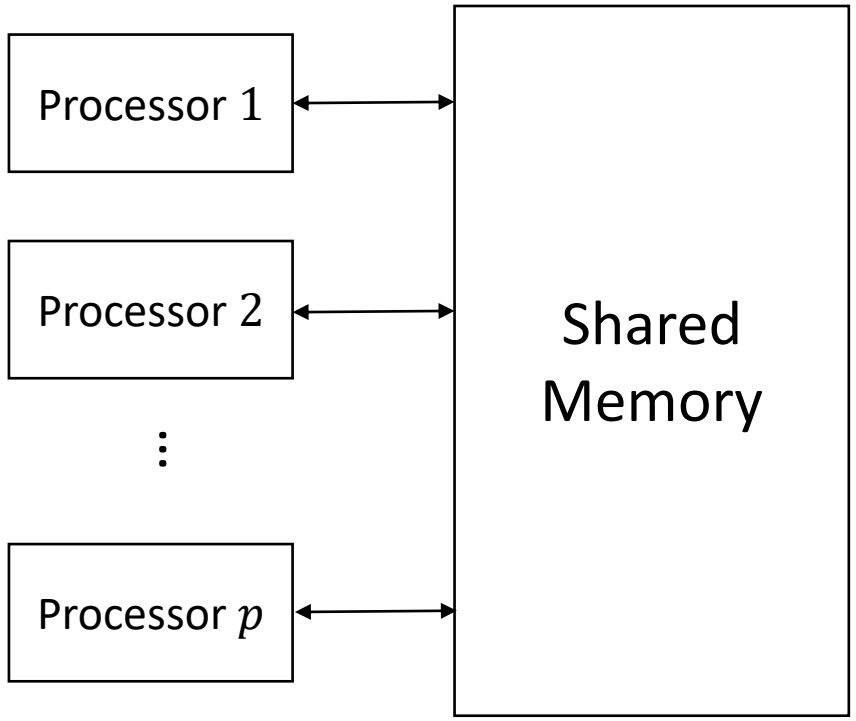
- **CREW**: Concurrent-Read-Exclusive-Write

  (simultaneous access only for read operations)

- **CRCW**: Concurrent-Read-Concurrent-Write

  (simultaneous access for both read and write operations)

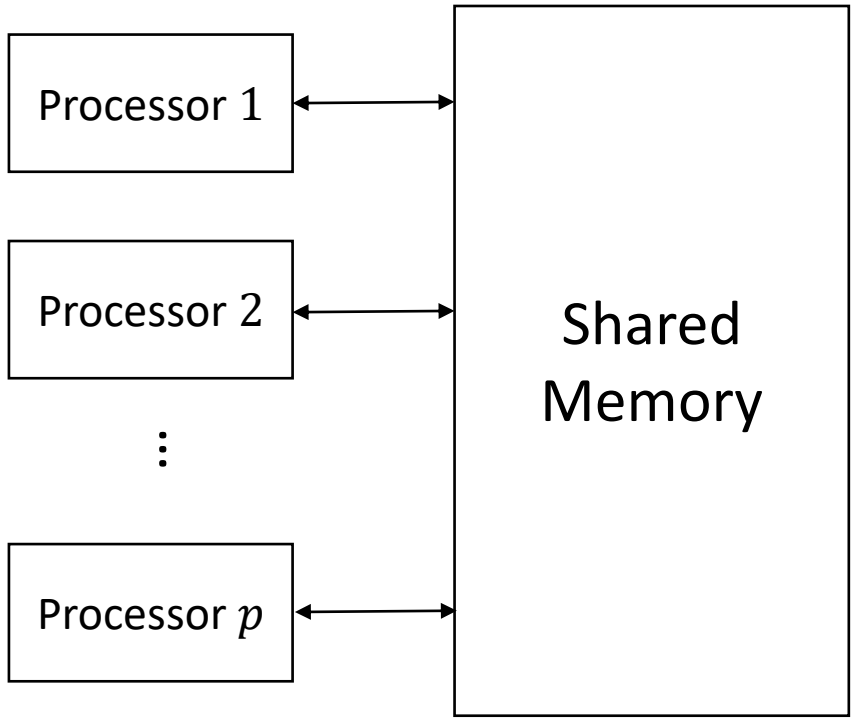
  - What happens when there are concurrent write operations?
    - **common**: all processors must write the same value
    - **arbitrary**: an arbitrary processor succeeds
    - **priority**: the processor with minimum id succeeds



OBSERVATION: An EREW algorithm is also a CREW algorithm. A CREW algorithm is also a CRCW algorithm.

Types of shared-memory models

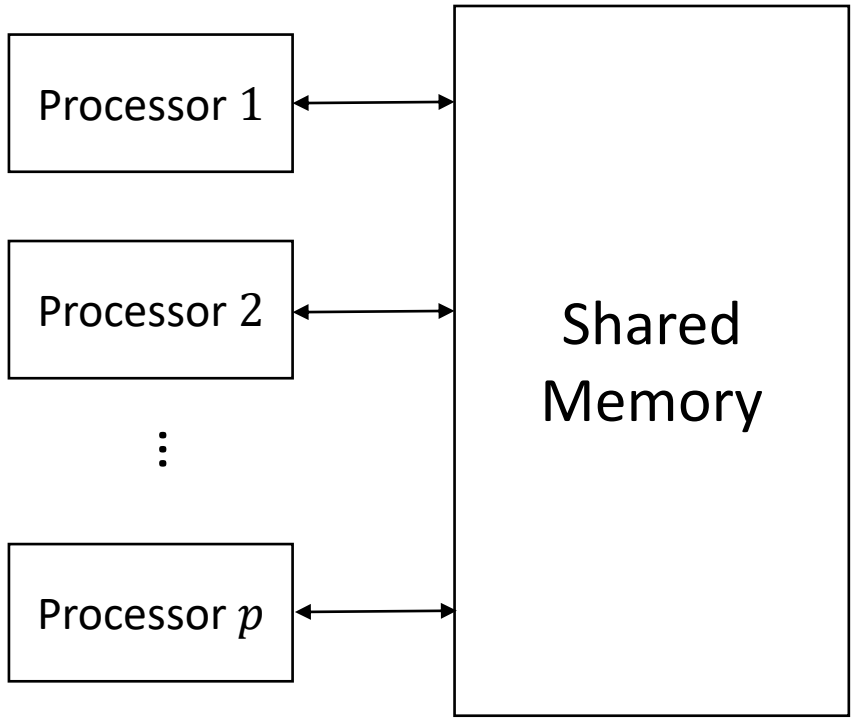
- Which program is executed by the processors?
  - **Multiple Instruction Multiple Data (MIMD)**
    Processors may run completely different programs
  - **Single Instruction Multiple Data (SIMD)**
    Processors run the same program, but the instructions that are executed and the data used may depend on the processor id



A famous shared-memory model: the PRAM

- In the **Parallel Random-Access Machine (PRAM)** model the processors operate **synchronously**
- Most of the algorithms developed for the PRAM model have been of **SIMD** type

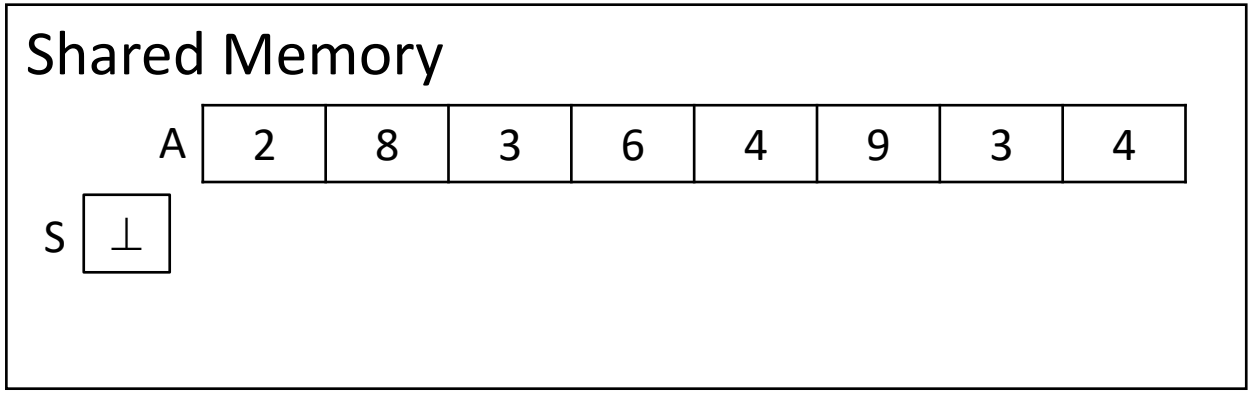
- We will denote shared variables with capitol letters (e.g., X) and local variables with small letters (e.g., x)



Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

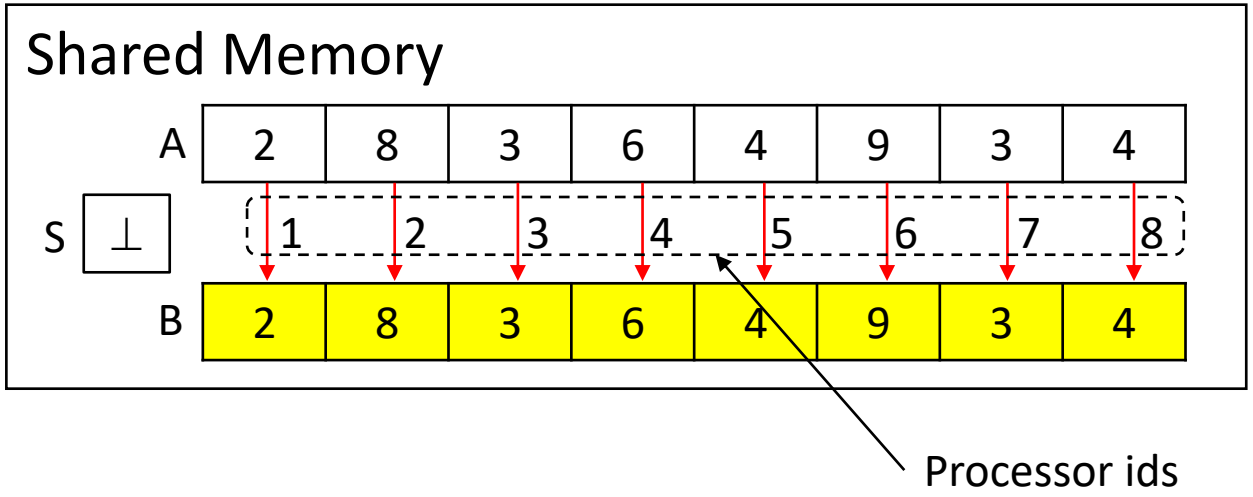


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

1. global-read(A[id], a)
2. global-write(a, B[id])

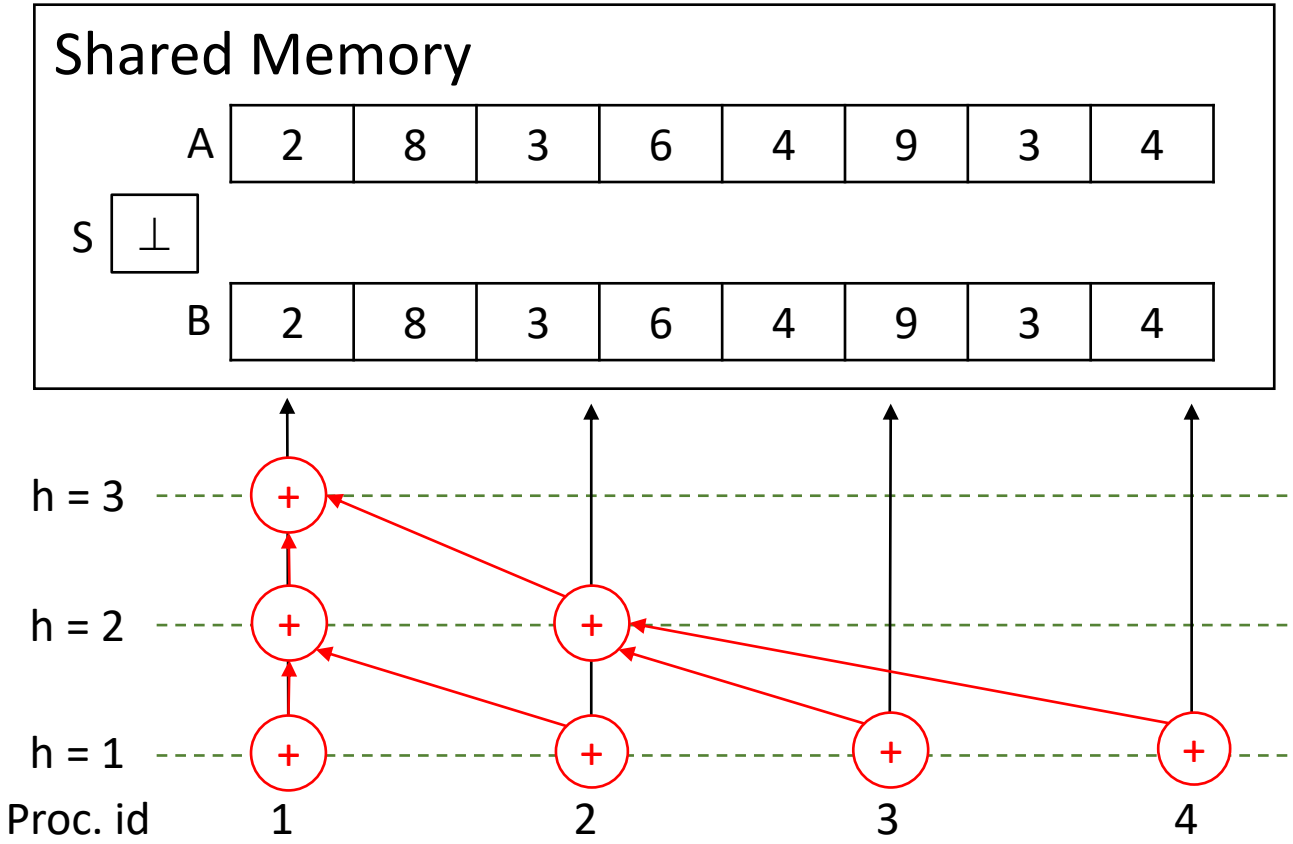


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

1. global-read(A[id], a)
2. global-write(a, B[id])

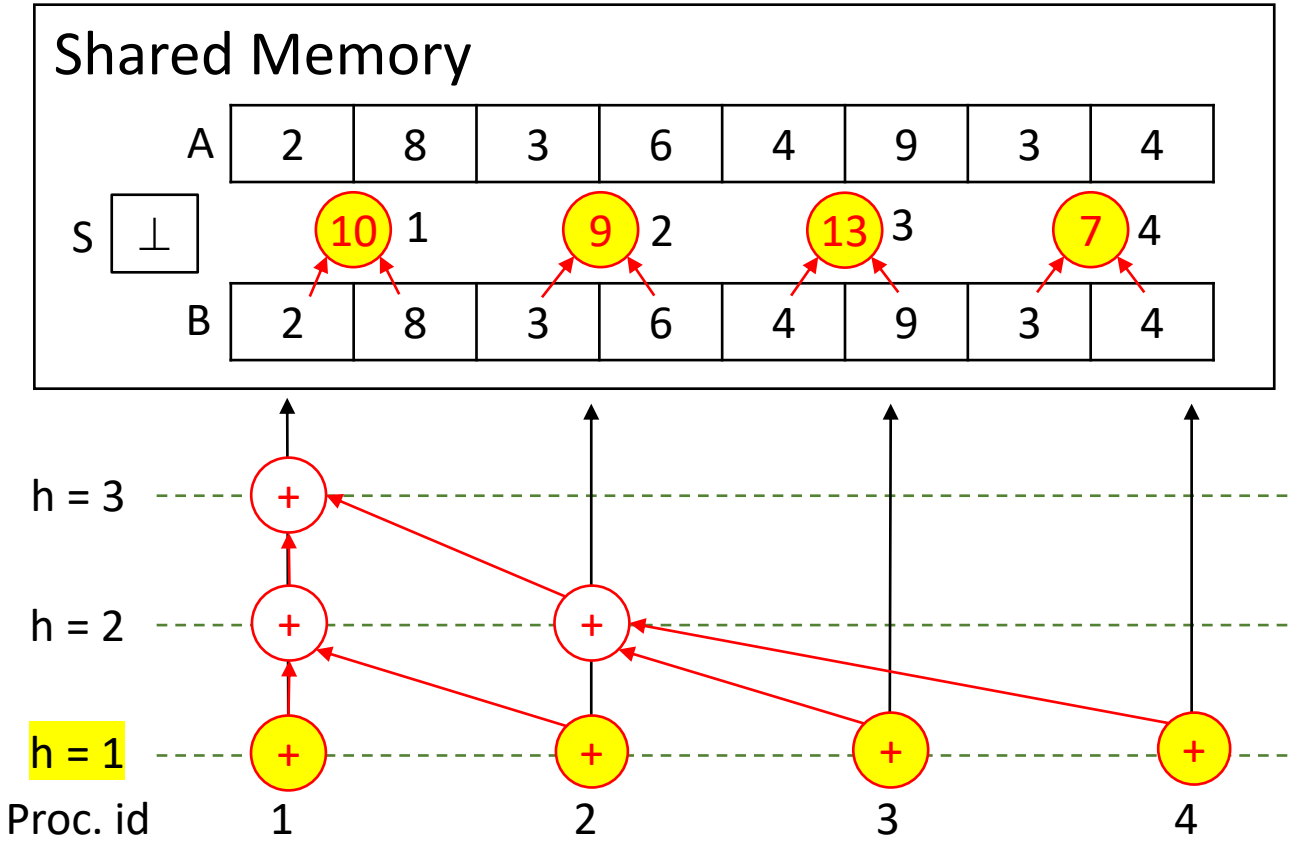


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
  if id ≤ n/2^h then
    global-read(B[2id – 1], x)
    global-read(B[2id], y)
    z ← x + y
    global-write(z, B[id])
```

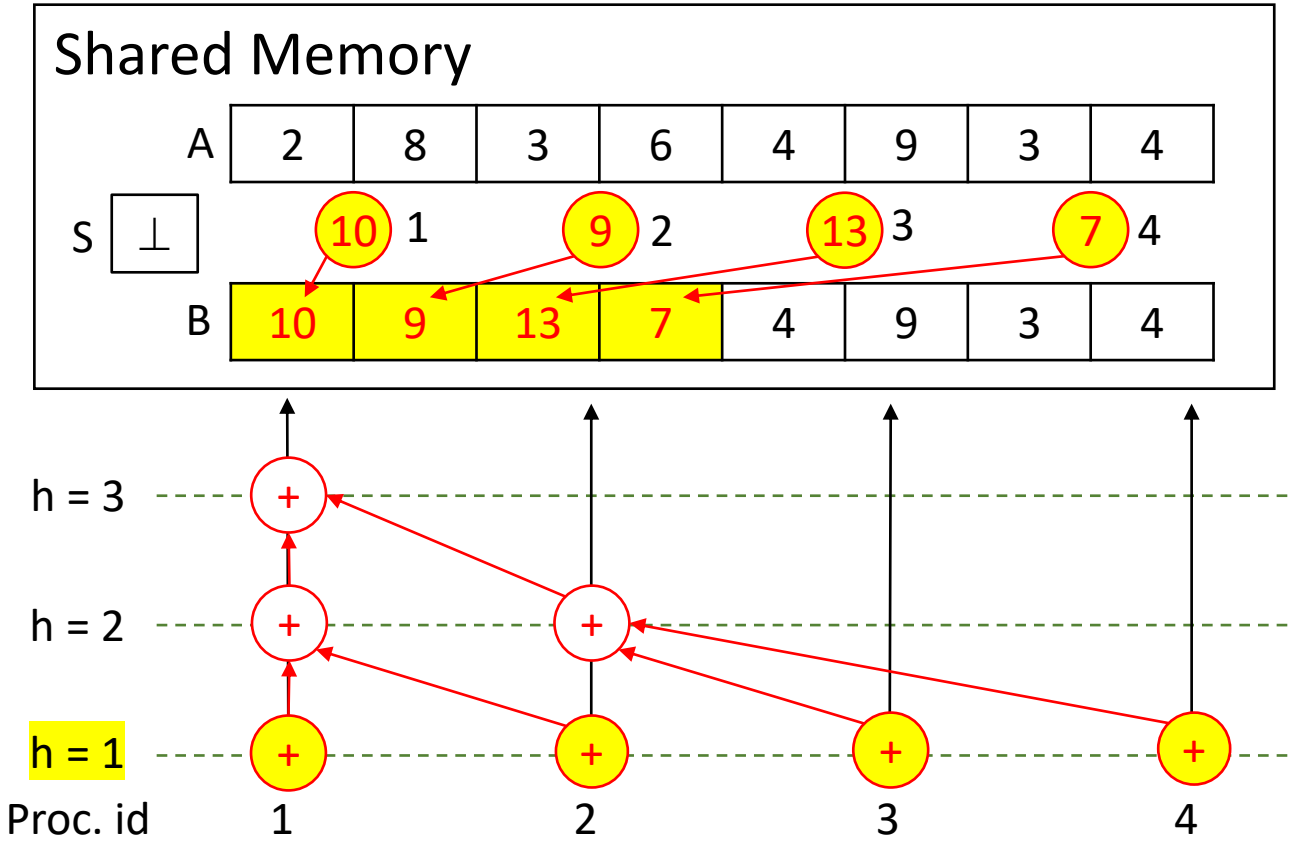


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
```

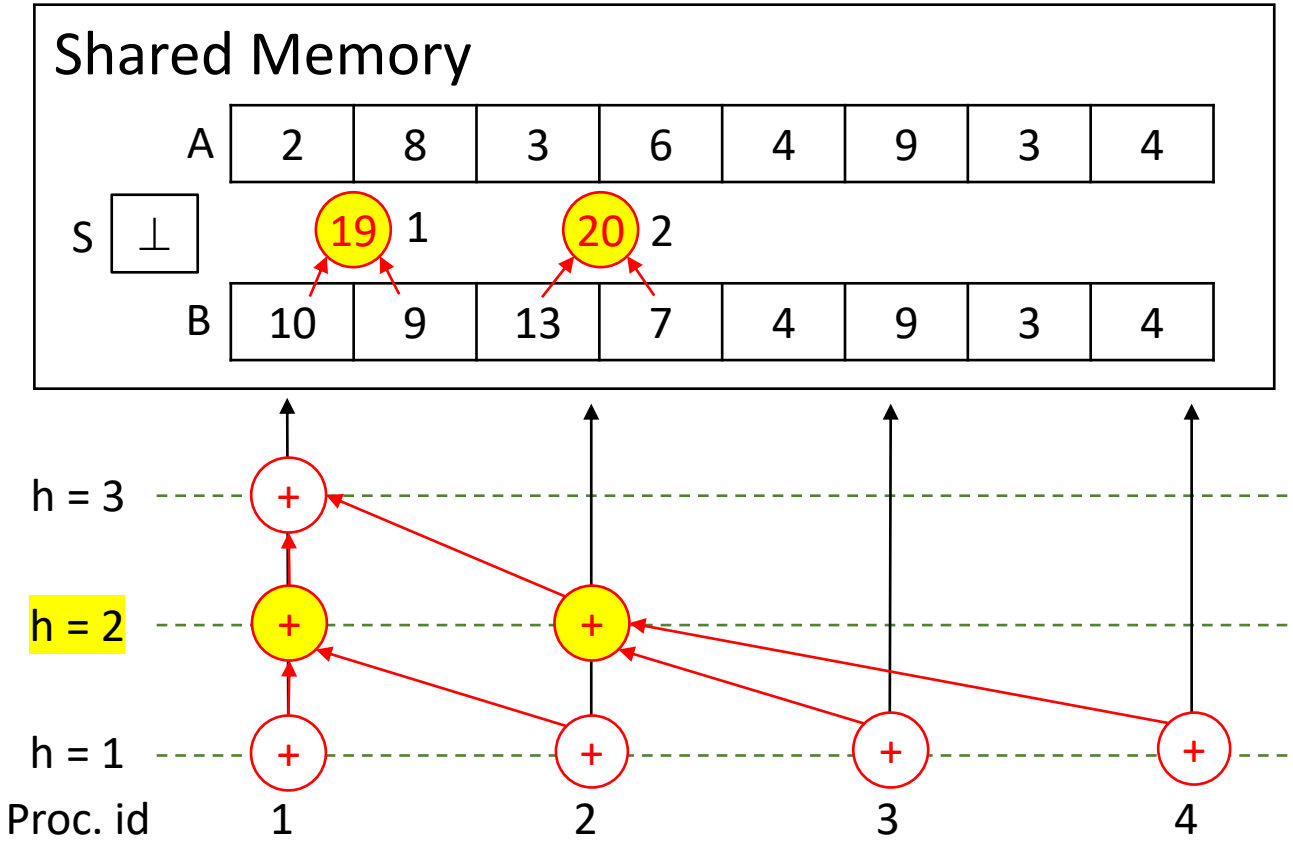


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
  if id ≤ n/2^h then
    global-read(B[2id – 1], x)
    global-read(B[2id], y)
    z ← x + y
    global-write(z, B[id])
```

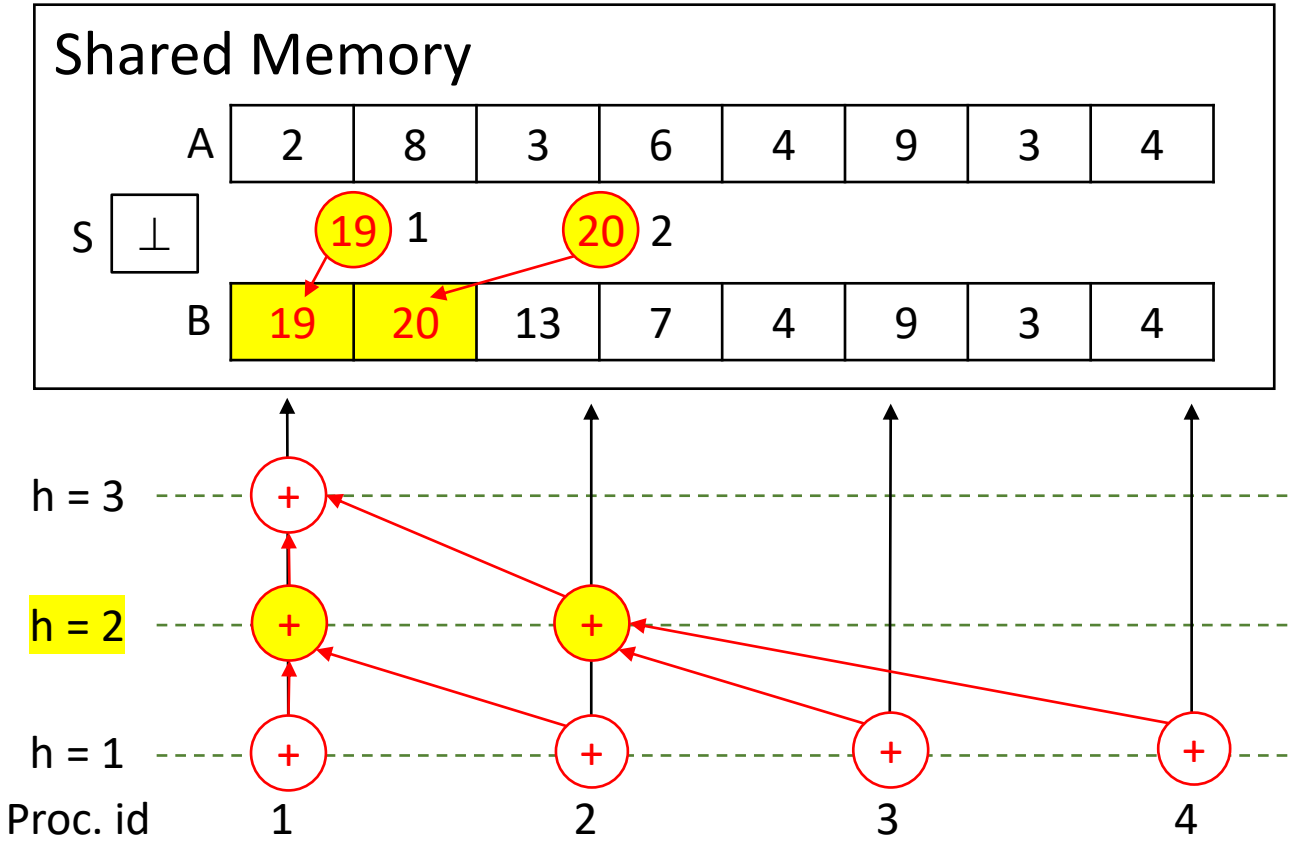


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
```

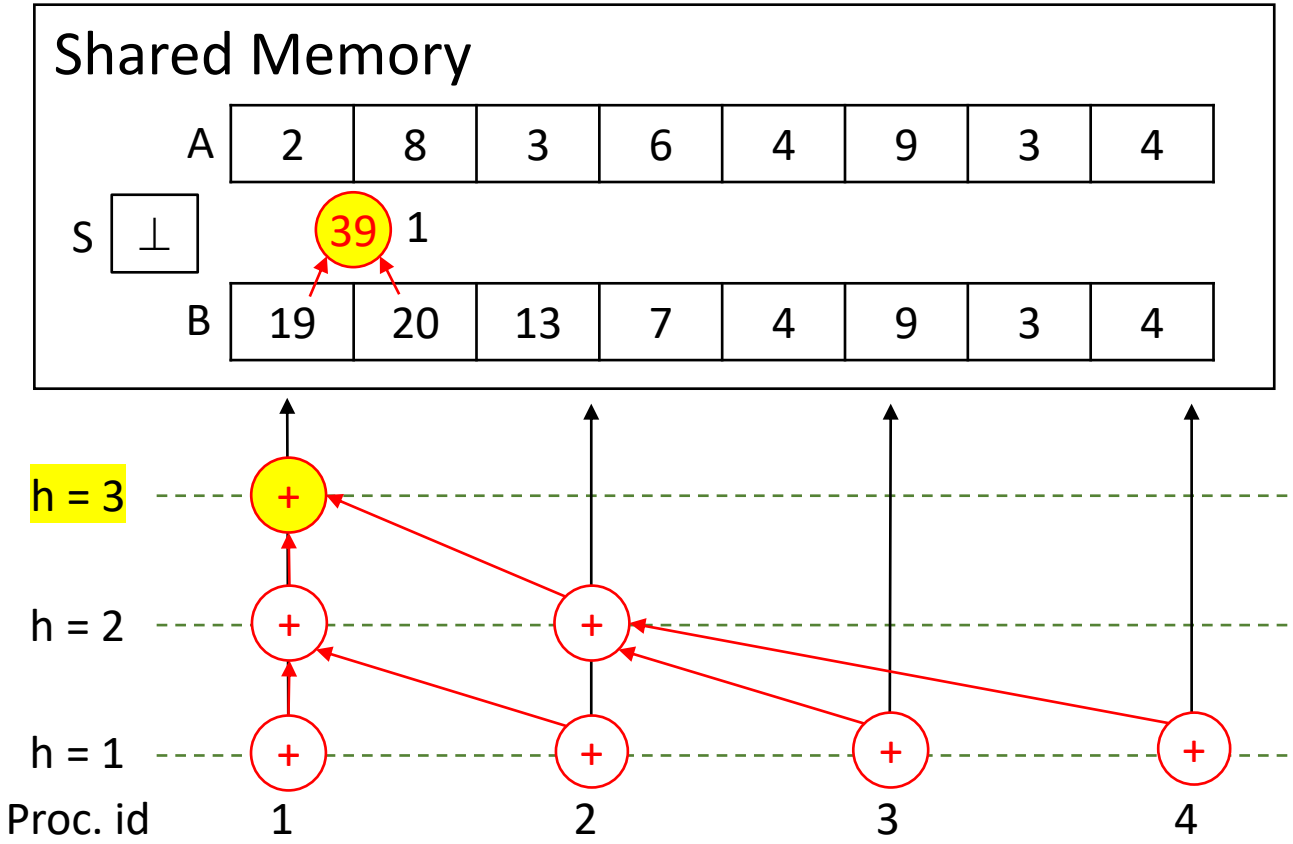


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

1. global-read(A[id], a)
2. global-write(a, B[id])
3. **for** h = 1 **to** $\log_2$ n **do**
4. **if** id $\leq n/2^h$ **then**
5. global-read(B[2id – 1], x)
6. global-read(B[2id], y)
7. z ← x + y

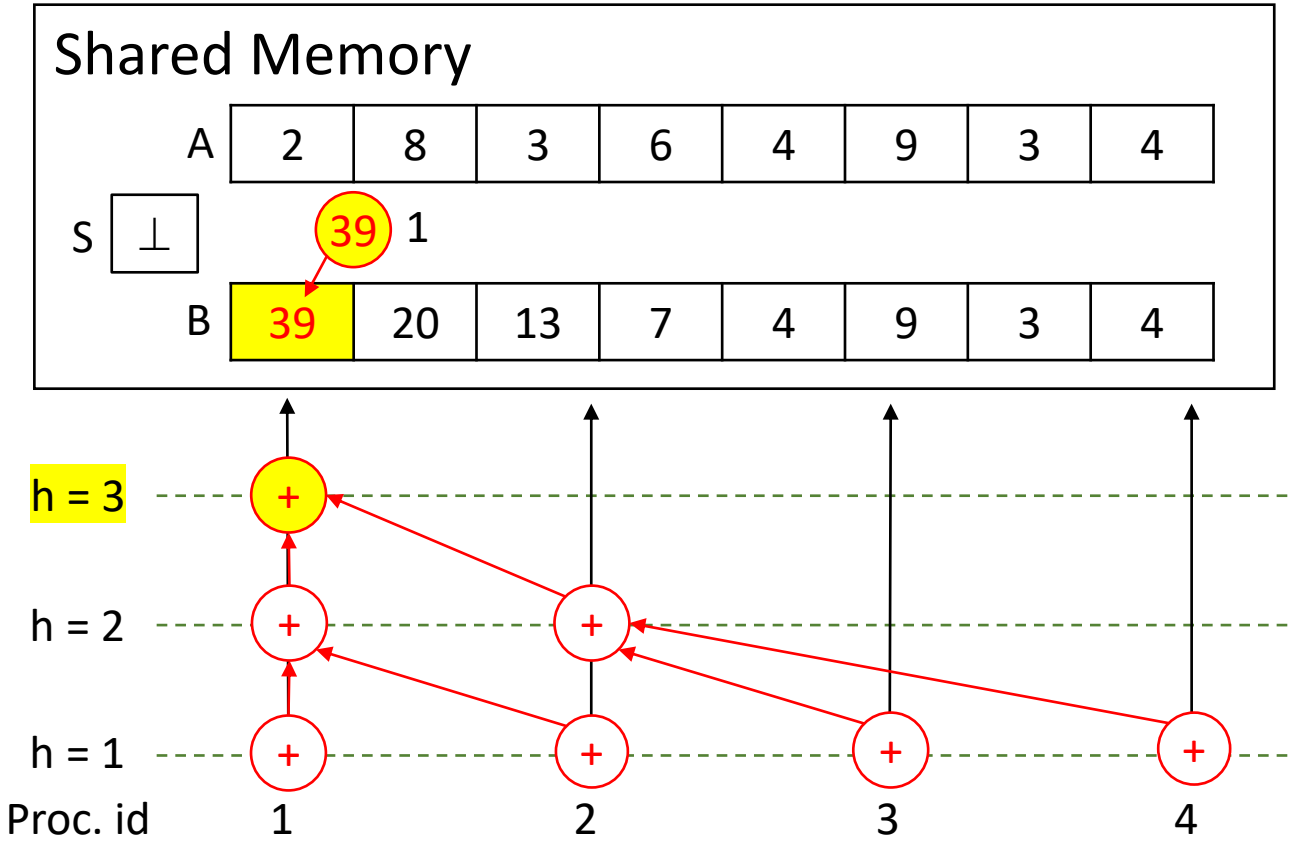


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
```

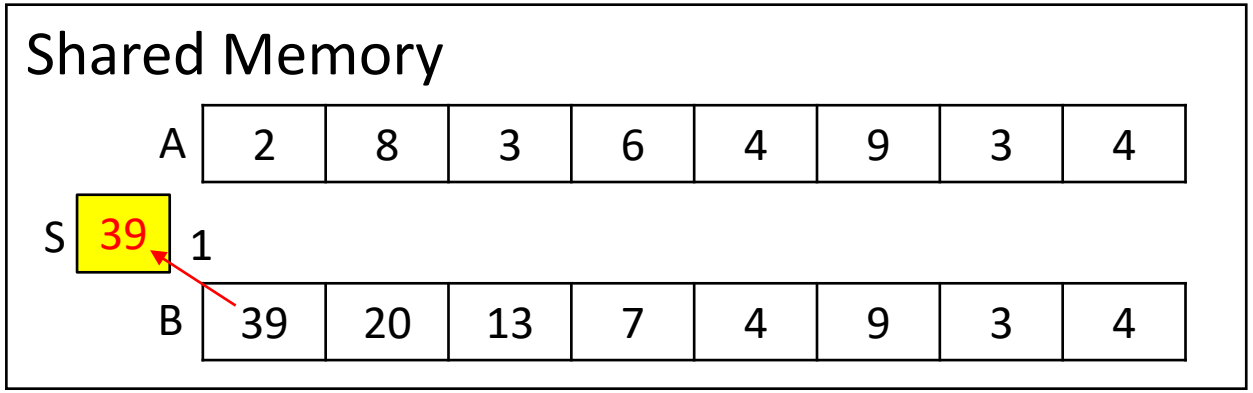


Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log_2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
if id = 1 then global-write(z, S)
```



Algorithm: Sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
if id = 1 then global-write(z, S)
```

- Concurrent write operations on same variable?
- NO
  - Only processor (with identifier) id writes into $B[id]$
  - Only processor 1 writes into S
- Concurrent read operations on same variable?
- NO
  - Only processor with identifier id reads $A[id]$
  - Only proc. id reads $B[2id - 1]$ and $B[2id]$

EREW PRAM Algorithm with $T_n(n) = O(\log n)$

Useful simplification of the pseudocode

- We simplify the pseudocode as shown below to avoid uninteresting details
- Example of storing, in the shared variable Z, the sum of two values stored in the shared variables X and Y, respectively

```
/* Extended code */
1.   global-read(X, x)
2.   global-read(Y, y)
3.   z ← x + y
4.   global-write(z, Z)
```

→

```
/* Simplified code */
Z ← X + Y
```

- We will use the above simplification of the code for every operator (not only +)

Algorithm: Sum of all n elements of an array (original pseudocode)

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
global-read(A[id], a)
global-write(a, B[id])
for h = 1 to log2 n do
    if id ≤ n/2^h then
        global-read(B[2id – 1], x)
        global-read(B[2id], y)
        z ← x + y
        global-write(z, B[id])
if id = 1 then global-write(z, S)
```

$B[id] \leftarrow A[id]$

$B[id] \leftarrow B[2id - 1] + B[2id]$

if id = 1 **then** $S \leftarrow B[id]$

Algorithm: Sum of all n elements of an array (simplified pseudocode)

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

```
B[id] ← A[id]
for h = 1 to log_2 n do
    if id ≤ n/2^h then
        B[id] ← B[2id – 1] + B[2id]
if id = 1 then S ← B[id]
```

The shared memory model

PROS

- Both synchronous and asynchronous models
- The PRAM model is more suitable to emphasize technique, paradigms, and methods, rather than detailed algorithms for specific applications
  - captures important parameters of parallel computation
  - no details concerning synchronization and communication
  - many algorithms for network models can be derived from algorithms on PRAM
  - it is still possible to incorporate issues about synchronization and communication

CONS

- The PRAM model hides many implementation details on synchronization and communication
  - Not easy to convert the algorithm into a real program

PRAM algorithms

Algorithm: Sum of all n elements of an array (simplified code)

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

/* We use p=n processors. The local variable id denotes the processor identifier */

1. B[id] ← A[id]
2. **for** h = 1 **to** $\log_2$ n **do**
3. **if** id ≤ $n/2^h$ **then**
4. B[id] ← B[2id – 1] + B[2id]
5. **if** id = 1 **then** S ← B[id]

- If we have $p = n$ processors, then $T_n(n) = O(\log n)$
- If we have $p = \infty$ processors, then $T_\infty(n) = O(\log n)$
  - No benefit in using more than n processors
- What happens if we have $p < n$ processors?
  - each processor i simulates, in $\approx n/p$ time steps, the parallel operations performed, in the actual code tailored for n processors, by processors with ids $(i-1)\cdot p+1, (i-1)\cdot p+2, \dots, i\cdot p$
  - $T_p(n) = O\left(\frac{n}{p}\log n\right)$
  - The speedup is $S_p(n) = O(p/\log n)$

GOAL: Design a PRAM algorithm that does not depend on the number p of processors.

The **work-time (WT) paradigm**

- informal guidelines for a two-level top-down description of a PRAM algorithm
  - Upper level: **suppresses** specific **details** of the algorithm
    - We do not need to specify how many processors there are
    - We do not have to allocate explicitly operations to processors
  - Lower level: **scheduling principle** which results in a full PRAM algorithm
- allows us to omit lower-level details that usually do not require any new ideas other than applying the scheduling principle

The upper level of the WT paradigm

- Describe the algorithms in terms of a sequence of time units, where each time unit may include any number of concurrent operations
- We use the following instruction
  - **for** h ≤ i ≤ k **pardo** statement
    means "statement is executed in parallel for all values of i in $\{h, \dots, k\}$"

DEFINITION: The *work* $W(n)$ of a PRAM algorithm is the worst-case total number of operations performed by the algorithm on input instances of size n.

EXAMPLE: sum of all n elements of an array

/* it stores the sum of all the $n=2^k$ elements of an array A in the shared variable S */

```
1.  for 1 ≤ i ≤ n pardo B[i] ← A[i]
2.  for h = 1 to log2 n do
3.    for 1 ≤ i ≤ n/2^h pardo
4.      B[i] ← B[2i – 1] + B[2i]
5.  S ← B[1]
```

The time is $T(n) = O(\log n)$

The work is $W(n) = O(n) + \sum_{h=1}^{\log_2 n} O\left(\frac{n}{2^h}\right) + O(1)$

$$= O(n) + O\left(\sum_{h=1}^{\log_2 n} \frac{n}{2^h}\right) + O(1)$$

$$= O(n)$$

The lower level of the WT paradigm

Brent's scheduling principle (1974): A PRAM algorithm with time $T(n)$ and work $W(n)$ can run on a p-processor PRAM in time $T_p(n) = O\left(\frac{W(n)}{p} + T(n)\right)$.

PROOF: Let $W_i(n)$ be the work performed by the algorithm during time step i.

Time unit						Number of operations (each • is an operation)
1	••••••••••	••••••••••	••••••••••	…	••••••••••	$W_1(n)$
2	••••	••••	••••	…	••••	$W_2(n)$
…	…	…	…	…	…	…
$T(n)$	••••••	••••••	••••••	…	••••••	$W_{T(n)}(n)$

The lower level of the WT paradigm

Brent's scheduling principle (1974): A PRAM algorithm with time $T(n)$ and work $W(n)$ can run on a p-processor PRAM in time $T_p(n) = O\left(\frac{W(n)}{p} + T(n)\right)$.

PROOF: Let $W_i(n)$ be the work performed by the algorithm during time step i.
For every i, each processor (except at most one of them) simulates $\left\lceil\frac{W_i(n)}{p}\right\rceil$ of the work $W_i(n)$ of time step i in $\left\lceil\frac{W_i(n)}{p}\right\rceil$ time steps.

Time unit	Processor 1	Processor 2	Processor 3	…	Processor p	Number of operations (each • is an operation)
1	• • • • • • • • •	• • • • • • • • •	• • • • • • • • •	…	• • • • • • • • •	$W_1(n)$
2	• • • •	• • • •	• • • •	…	• • • •	$W_2(n)$
…	…	…	…	…	…	…
$T(n)$	• • • • • •	• • • • • •	• • • • • •	…	• • • • • •	$W_{T(n)}(n)$

The lower level of the WT paradigm

Brent's scheduling principle (1974): A PRAM algorithm with time $T(n)$ and work $W(n)$ can run on a p-processor PRAM in time $T_p(n) = O\left(\frac{W(n)}{p} + T(n)\right)$.

PROOF: Let $W_i(n)$ be the work performed by the algorithm during time step i.
For every i, each processor (except at most one of them) simulates $\left\lceil \frac{W_i(n)}{p} \right\rceil$ of the work $W_i(n)$ of time step i in $\left\lceil \frac{W_i(n)}{p} \right\rceil$ time steps.
The time taken by the corresponding p-processor PRAM algorithm is at most

$$\sum_{i=1}^{T(n)} \left\lceil \frac{W_i(n)}{p} \right\rceil \leq \sum_{i=1}^{T(n)} \left(\frac{W_i(n)}{p} + 1 \right) = \frac{W(n)}{p} + T(n)$$

OBSERVATION: The success of this principle depends on two implementation issues:

1. calculation of $W_i(n)$ for every i (usually trivial)
2. the partitioning of $W_i(n)$ evenly among the p processors (not trivial)
   (i.e., allocation of each processor to the appropriate operations and operands)

An example of Brent's scheduling principle: sum of all n elements of an array

```
/* it stores the sum of all the n=2^k
elements of an array A in the shared
variable S */
1.    for 1 ≤ i ≤ n pardo B[i] ← A[i]
2.    for h = 1 to log_2 n do
3.      for 1 ≤ i ≤ n/2^h pardo
4.        B[i] ← B[2i – 1] + B[2i]
5.    S ← B[1]
```

```
/* This is the code executed by processor with identifier id */
1.    l ← n/p
2.    for j = 1 to l do B[l(id – 1)+ j] ← A[l(id – 1)+ j]
3.    for h = 1 to log_2 n do
4.      if (k – h – q) ≥ 0 then
5.        for j = (id – 1)2^(k–h–q)+ 1 to id · 2^(k–h–q) do B[j] ← B[2j-1]+B[2j]
6.      else if id ≤ 2^(k–h) then B[id] ← B[2id – 1] + B[2id]
7.    if id = 1 then S ← B[id]
```

EXAMPLE: Assume p divides n, i.e., $p = 2^q$ with $q \leq k$. Let $l = \frac{n}{p} = 2^{k-q}$. If p does not divide n, then $l \leftarrow \lceil n/p \rceil$ and we need additional tests to check that we are in the array boundaries)

- Divide A into p subarrays and make processor i responsible of subarray $A[l(i-1)+1..li]$
- At each level h of the balanced binary tree scheme, use same trick to generate B
  - the number of possible concurrent operations at level h is $\frac{n}{2^h} = 2^{k-h}$
  - If $p \leq 2^{k-h}$ then we divide the operations equally among the processors
  - If $p > 2^{k-h}$ then the 2^{k-h} processors with lowest ids execute these operations

The optimality notion

DEFINITION: A PRAM algorithm described with the WT paradigm for problem Π is **(work) optimal** if the work $W(n)$ required by the algorithm satisfies $W(n) = T^*(n)$, where $T^*(n)$ is the time complexity of best sequential algorithm solving Π.

OBSERVATION: A work optimal PRAM algorithm with $T(n)$ time can be simulated on a p-processor PRAM in time $T_p(n) = O\left(\frac{T^*(n)}{p} + T(n)\right)$ using Brent's scheduling principle.

The speedup is $S_p(n) = \frac{T^*(n)}{T_p(n)} = \Omega\left(\frac{T^*(n)}{\frac{T^*(n)}{p}+T(n)}\right) = \Omega\left(\frac{p \cdot T^*(n)}{T^*(n)+p \cdot T(n)}\right)$.

The algorithm achieves an optimal speedup $S_p(n) = \Theta(p)$ when $p = \Theta\left(\frac{T^*(n)}{T(n)}\right)$.

DEFINITION: Given a problem Π, a <u>work optimal</u> PRAM algorithm described with the WT paradigm for Π is **time optimal** if the parallel time $T(n)$ cannot be improved by any other <u>work optimal</u> PRAM algorithm.

Check you understanding

The shared memory of an EREW PRAM contains a Boolean array X of size $n = 2^k$. Given $p = 2^q$ processors, where $q \leq k$, develop an algorithm that has to be executed by each processor to compute the Boolean AND of the x_i's. Your algorithm must run in $O\left(\frac{n}{p} + \log\frac{n}{p}\right)$ time. Show how to perform the same computation in $O(n/p)$ time on the common CRCW PRAM.

Check you understanding

Show how to broadcast the value contained in the global variable X to all the local memories of the p processors of a EREW PRAM in $O(\log p)$ time. Determine how much time it takes to perform the same operation on the CREW PRAM. Finally, prove that any EREW PRAM time-optimal algorithm needs $\Omega(\log p)$ time.